

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Уральский федеральный университет имени первого Президента России
Б.Н. Ельцина» (УрФУ)
Институт радиоэлектроники и информационных технологий – РтФ

Отчёт по заданию

**«Симуляция физического явления в среде Unity:
столкновения ионов на Большом адронном коллайдере»**

По дисциплине «Цифровые двойники»

*«Чтобы увидеть самое малое во Вселенной,
человечеству пришлось построить самое
большое»*

Студент группы РИЗ-151105у
направления радиотехника
Токарев Артём Сергеевич

Екатеринбург
2026

КОНСПЕКТ ЛЕКЦИИ «UNITY: ОТ ИГРОВОГО ДВИЖКА К ВИРТУАЛЬНОМУ ЭКСПЕРИМЕНТУ»

Unity — это среда разработки интерактивных трёхмерных приложений, появившаяся в 2005 году и сегодня занимающая одну из лидирующих позиций среди игровых движков. Однако сводить Unity только к играм было бы неверно: те же механизмы, что оживляют игровые миры, позволяют ставить виртуальные эксперименты — от тренажёров для пилотов до визуализации процессов, которые невозможно увидеть глазами: распространения волн, движения зарядов, столкновений элементарных частиц.

Ключевая идея архитектуры Unity — разделение «сущности» и «поведения». Сцена состоит из объектов `GameObject`, каждый из которых получает свойства через присоединяемые компоненты: `Transform` отвечает за положение в пространстве, `Renderer` — за отображение, `Collider` и `Rigidbody` — за участие в физике. Программист не переписывает объект целиком, а комбинирует готовые блоки и дописывает собственные компоненты — скрипты на языке `C#`, наследуемые от класса `MonoBehaviour`.

Работа скрипта подчинена циклу кадров. Метод `Start()` выполняется один раз при запуске, `Update()` — каждый кадр, `FixedUpdate()` — с постоянным шагом по времени, что принципиально для корректного физического моделирования. Величина `Time.deltaTime` содержит время, прошедшее с предыдущего кадра: благодаря ей движение объектов описывается через приращения координат, то есть фактически выполняется пошаговое численное решение уравнений движения. Получается, что любая симуляция в Unity — это дискретная модель непрерывного процесса, и точность модели определяется шагом по времени.

Для отображения результата Unity предлагает несколько конвейеров рендеринга (`Built-in`, `URP`, `HDRP`), систему пользовательского интерфейса (`Canvas`, кнопки, текст), а также вспомогательные средства визуализации — `LineRenderer` для траекторий и линий, `TrailRenderer` для следов движущихся

объектов, источники света. Физическое ядро движка (PhysX) берёт на себя обнаружение столкновений и расчёт динамики твёрдых тел.

Вывод: Unity удобно рассматривать как конструктор виртуальных лабораторий. Если явление можно описать уравнениями и нарисовать, его можно смоделировать в Unity — и именно наглядность, недостижимая на бумаге, делает движок ценным инструментом для изучения физики.

РЕФЛЕКСИЯ ПО ПРОЙДЕННОМУ МАТЕРИАЛУ

Для практической части я выбрал тему, которая давно вызывала у меня любопытство, — работу Большого адронного коллайдера. До этого мои представления о ЦЕРН были на уровне научно-популярных роликов: «разгоняют и сталкивают частицы». Когда пришлось превращать это в работающую модель, выяснилось, сколько деталей скрывается за простой формулировкой: пучки нужно сначала инжектировать в кольцо, затем разгонять, удерживая магнитным полем, и только потом сводить в точке взаимодействия внутри детектора.

Самым интересным открытием для меня стала связь энергии пучка с зарядом и массой ядра. Магниты ускорителя удерживают частицу тем сильнее, чем больше её заряд, поэтому энергия на один нуклон пропорциональна отношению Z/A : протон в LHC получает 6,8 ТэВ, а ядро свинца — лишь около 2,7 ТэВ на нуклон. Заложив эту зависимость в код, я с удивлением получил для столкновений свинца энергию 5,36 ТэВ — ровно ту цифру, что указана в официальных материалах ЦЕРН. Момент, когда твоя модель сама «выдаёт» реальное значение, убеждает в правильности понимания лучше любой проверки по учебнику.

Вторая важная вещь, которую я осознал, — роль магнитного поля в детекторе. Треки заряженных частиц закручиваются по окружностям, радиус которых пропорционален импульсу, а направление закрутки определяется знаком заряда. То есть кривизна трека — это не просто красивая картинка, а измерительный инструмент: по ней физики определяют, что за частица родилась. В симуляции я добавил выключатель поля, и контраст между «закрученной» и «прямой» картинками оказался очень наглядным.

Из трудностей отмечу организацию интерфейса пульта управления и состояния ускорителя: цикл «инжекция — разгон — стабильные пучки — соударение — сброс» пришлось оформить конечным автоматом, и я впервые на практике почувствовал, зачем эта конструкция нужна в программировании.

Главный вывод: моделирование заставляет разбираться в явлении до конца. Пока пишешь код, невозможно отделаться общими словами — каждая формула должна заработать. Думаю, такой способ изучения физики — через построение собственной модели — я буду применять и к другим темам курса.

РАЗРАБОТАННЫЙ ПРОЕКТ И ССЫЛКА НА РЕПОЗИТОРИЙ

В программном пакете Unity (версия 6000.4) разработана интерактивная симуляция работы Большого адронного коллайдера с пультом оператора. Пользователь выбирает сорта ионов для двух встречных пучков (16 ядер, реально используемых в ускорителях, — от протона до урана-238) и запускает цикл ускорителя: инжекция, разгон, стабильные пучки, соударения, сброс пучка.

В модели учтены: зависимость энергии пучка от отношения заряда к массовому числу ядра $E = 6,8 \cdot (Z/A)$ ТэВ на нуклон (для столкновений свинец–свинец воспроизводится реальное значение $\sqrt{s} = 5,36$ ТэВ); рост множественности рождённых частиц с массой сталкивающихся ядер; искривление треков заряженных частиц магнитным полем детектора (радиус кривизны пропорционален импульсу, направление закрутки определяется знаком заряда); образование кварк-глюонной плазмы при столкновениях тяжёлых ионов; вылет осколков-спектаторов вдоль направления пучка. Интерфейс выполнен в виде пульта управления с журналом операций, статусной панелью и сводкой каждого зарегистрированного события.

Ссылка на репозиторий проекта:

<https://github.com/REMOOROO/Simulator-CERN.git>

В репозитории размещены исходный код симуляции (Assets/Scripts/ColliderSimulation.cs) и файл readme.md с описанием физики модели, элементов пульта управления и инструкцией по запуску.